

Software Requirements Specification

SkillSim – A Browser-Based Hands-On Lab Learning Platform

(IEEE 29148 Based)

Vivek Reddy and Phani Chandan Reddy
Department of Computer Science
vivekreddy9036@gmail.com

Version 1.0 – August 13, 2026

Abstract

This Software Requirements Specification (SRS) describes SkillSim, a browser-based, hands-on lab platform modeled on the KodeKloud learning experience. Students work through auto-validated, step-by-step labs inside real, ephemeral sandbox containers reached through an in-browser terminal, with each step's completion verified by executing a validation script inside the student's own container rather than by self-report. This document follows the IEEE 29148 SRS structure and covers scope, functional and non-functional requirements, interfaces, data model, use cases, analysis models, security requirements, risk analysis, a requirement traceability matrix, and acceptance criteria for the project's five-week, single-VM, no-Kubernetes release.

Contents

1	Introduction	4
1.1	Purpose	4
1.2	Scope	4
1.3	Definitions, Acronyms and Abbreviations	4
1.4	References	4
1.5	Document Overview	4
2	Overall Description	5
2.1	Product Perspective	5
2.2	Product Functions	5
2.3	User Classes and Characteristics	5
2.4	Operating Environment	5
2.5	Design and Implementation Constraints	5
2.6	Assumptions and Dependencies	5
3	Functional Requirements	6
4	Non-Functional Requirements	9
4.1	Performance	9
4.2	Security	9
4.3	Reliability	9
4.4	Availability	9
4.5	Usability	9
4.6	Maintainability	9
4.7	Portability	9
5	External Interface Requirements	9
5.1	User Interface	9
5.2	Hardware Interface	10
5.3	Software Interface	10
5.4	Communication Interface	10
6	Data Requirements	10
6.1	Entities	10
6.2	Attributes	10
6.3	Relationships	10
6.4	Validation Rules	10
6.5	Database Requirements	11
7	Use Cases	11
8	Analysis Models	13
8.1	Use Case Diagram	13
8.2	Activity Diagram	14
8.3	Sequence Diagram	15
8.4	Class Diagram	16
8.5	ER Diagram / Data Model	16

9	Security Requirements	16
9.1	Authentication	16
9.2	Authorization	17
9.3	Input Validation	17
9.4	Encryption	17
9.5	Logging and Auditing	17
9.6	Session Management	17
9.7	Backup and Recovery	17
10	Risk Analysis	17
11	Requirement Traceability Matrix (RTM)	18
12	Acceptance Criteria	19
13	Appendices	19
13.1	Glossary	19
13.2	References	19
13.3	Revision History	20

1 Introduction

1.1 Purpose

This SRS defines the functional and non-functional requirements of SkillSim, a hands-on labs platform for technical skill practice. It guides implementation, serves as a contract between the project team and course stakeholders, and is a durable reference for extending the system beyond its initial five-week scope.

1.2 Scope

SkillSim lets a student browse a catalog of technology *tracks* (e.g., Docker, Linux), select a *lab* within a track, and work through an ordered sequence of *steps* inside a real, ephemeral sandbox container reachable through an in-browser terminal. Each step ships with a validation script; when a student believes a step is complete, the platform executes that script inside the student's live container and automatically unlocks the next step only on success. Progress and points are persisted per user. The initial release targets a single host VM, a hand-authored (non-CMS) content pipeline, and excludes Kubernetes-based orchestration, multi-node scaling, and third-party LMS integration.

1.3 Definitions, Acronyms and Abbreviations

- **Track** – A named grouping of related labs (e.g., "Docker").
- **Lab** – A single hands-on exercise composed of ordered steps.
- **Step** – One unit of instruction within a lab, paired with a validation script.
- **Sandbox** – An ephemeral, hardened Docker container spawned for one user's lab session.
- **JWT** – JSON Web Token; a short-lived, `sandbox:attach`-scoped credential authorizing a WebSocket terminal connection to one specific session.
- **DinD** – Docker-in-Docker; a sandbox running its own nested Docker daemon, used only by labs that teach Docker itself.
- **app-backend** – The REST/database-owning service.
- **terminal-server** – The service that owns the Docker socket and bridges WebSocket terminal I/O.
- **RTM** – Requirement Traceability Matrix.

1.4 References

1. ISO/IEC/IEEE 29148:2018, *Systems and software engineering – Life cycle processes – Requirements engineering*.
2. IEEE Std 830-1998, *IEEE Recommended Practice for Software Requirements Specifications*.
3. Docker Inc., *Docker Engine API Reference*, docs.docker.com.
4. KodeKloud, *Hands-on Labs Platform* (UX reference), kodekloud.com.
5. Prisma Data Platform, *Prisma ORM Documentation*, prisma.io/docs.
6. OWASP Foundation, *OWASP Top Ten*, owasp.org.

1.5 Document Overview

Section 2 describes the product at a high level. Section 3 lists functional requirements as individual cards. Section 4 covers non-functional requirements. Section 5 defines external interfaces. Section 6 defines the data model. Section 7 specifies use cases. Section 8 presents five analysis-model diagrams. Section 9 details security requirements. Section 10 is a risk analysis. Section 11 is the requirement traceability matrix. Section 12 gives measurable acceptance criteria. Section 13 contains appendices.

2 Overall Description

2.1 Product Perspective

SkillSim is a new, self-contained system. It is composed of three deployable units sharing one host VM during the class-project phase: a single-page *frontend*, a REST *app-backend* that owns the relational database, and a *terminal-server* that exclusively owns the Docker socket and all container lifecycle operations. This separation confines privileged Docker access to one process, so a compromised or buggy REST endpoint cannot directly manipulate containers.

2.2 Product Functions

- Register, authenticate, and browse a catalog of tracks and labs.
- Start a lab session, provisioning an ephemeral sandbox container scoped to that session.
- Provide an interactive terminal into the sandbox over WebSocket.
- Present step-by-step lesson content beside the terminal.
- Auto-validate each step by executing a validation script inside the student's own container, unlocking the next step only on success.
- Persist per-user progress and award points on success.
- Let content authors publish labs as plain files, without a CMS.

2.3 User Classes and Characteristics

- **Student (primary actor):** Browser only; basic command-line familiarity assumed.
- **Content Author (project team):** Authors labs as files under `labs/<track>/<lab-slug>/` and runs a sync script; needs git/shell access, not a special UI.
- **System actor – terminal-server:** Not human; spawns, execs into, and reaps sandbox containers on students' behalf.

2.4 Operating Environment

A single Linux VM with a working Docker Engine installation. The app-backend and terminal-server are Node.js/TypeScript services; persistent state lives in PostgreSQL. The frontend is a static SPA (Vite/React) served to any evergreen desktop browser with WebSocket support. No Kubernetes or multi-host scheduling is in scope.

2.5 Design and Implementation Constraints

- **Timeline:** Five-week class-project schedule; scope is fixed, not expanded mid-project.
- **Single VM:** All services and the database co-locate on one host.
- **No Kubernetes:** Sandboxes are scheduled directly against the local Docker Engine.
- **File-based content only:** No admin UI/CMS for authoring labs.
- **Auto-validation only:** Steps advance strictly via a passing validation script – manual "mark as done" is out of scope.

2.6 Assumptions and Dependencies

The host VM has sufficient CPU/memory to run several concurrent ephemeral containers; Docker images for each lab are pre-built and tagged before publishing; students have a modern browser. The system depends on the Docker Engine API, PostgreSQL, and standard JWT libraries; no third-party SaaS dependency is required at runtime.

3 Functional Requirements

Each requirement below follows the ID / Title / Description / Inputs / Processing / Outputs / Priority / Acceptance Criteria format. Priority uses MoSCoW (Must/Should/Could).

Requirement ID: FR-01 **Title:** User Registration

Description: The system shall allow a new user to register with an email, password, and display name.

Inputs: Email, password, display name.

Processing: Validate email format and uniqueness; hash password; create **User** row.

Outputs: Created user account; redirect to catalog.

Priority: Must

Acceptance Criteria: A new, unique email registers successfully in under 2 seconds; a duplicate email is rejected with a clear error.

Requirement ID: FR-02 **Title:** User Login

Description: The system shall authenticate registered users using email and password.

Inputs: Email, password.

Processing: Look up user by email; verify password hash; issue session credential.

Outputs: Authenticated session; user redirected to dashboard/catalog.

Priority: Must

Acceptance Criteria: Valid users are authenticated within 2 seconds; invalid credentials are rejected without revealing which field was wrong.

Requirement ID: FR-03 **Title:** Browse Tracks and Labs

Description: The system shall list all tracks and, per track, the labs it contains, including title, difficulty, and duration.

Inputs: Track slug (optional, for detail view).

Processing: Query **Track** and related **Lab** rows ordered by **orderInTrack**.

Outputs: Rendered catalog / track-detail page.

Priority: Must

Acceptance Criteria: Catalog page loads all tracks in under 1 second on the reference dataset; track detail lists labs in the authored order.

Requirement ID: FR-04 **Title:** Start Lab Session

Description: The system shall provision an ephemeral sandbox container from the lab's pre-built image and issue a session-scoped JWT when a student starts a lab.

Inputs: User ID (from session), lab slug.

Processing: Create **UserLabSession** row; embed step validation-script filenames in JWT claims; request container spawn.

Outputs: Sandbox JWT (**sandbox:attach** scope); active session record.

Priority: Must

Acceptance Criteria: A lab start results in a running container and a valid JWT within a few seconds; the JWT is rejected if reused for a different session.

Requirement ID: FR-05 **Title:** Interactive Terminal Session

Description: The system shall open an interactive terminal into the student's sandbox over WebSocket, verifying the JWT and request Origin before upgrade.

Inputs: Sandbox JWT, WebSocket upgrade request, keystrokes, resize events.

Processing: Verify JWT + Origin; `docker exec` with `Tty:true`; bridge binary frames bidirectionally.

Outputs: Live terminal I/O; resized PTY on window resize.

Priority: Must

Acceptance Criteria: A valid JWT opens a working shell within 2 seconds; an invalid or expired JWT is rejected before any container access occurs.

Requirement ID: FR-06 **Title:** Check Step (Auto-Validation)

Description: The system shall execute the current step's validation script inside the student's own container when the student triggers a Check action, and report pass/fail.

Inputs: `check_step` control frame identifying the current step.

Processing: Exec the step's `.check.sh` script inside the session's container; capture exit code.

Outputs: `step_result:{passed:boolean}` sent back over the WebSocket.

Priority: Must

Acceptance Criteria: A correctly completed step returns `passed:true`; an incomplete step returns `passed:false` without crashing the session.

Requirement ID: FR-07 **Title:** Unlock Next Step and Award Points

Description: The system shall unlock the next step and award points only after the current step's validation script reports success.

Inputs: Passing `step_result` from FR-06.

Processing: `terminal-server` calls `/internal/progress`; `app-backend` updates `UserStepProgress.status` and `User.points`.

Outputs: Persisted progress row; updated point total; unlocked next step in the UI.

Priority: Must

Acceptance Criteria: After a passing check, `UserStepProgress.status=completed` and the point increment are visible in the database and reflected in the UI without a page reload.

Requirement ID: FR-08 **Title:** Idle Session Reaping

Description: The system shall idle-reap sandbox containers and mark sessions expired after a period of client inactivity.

Inputs: Session last-activity timestamp.

Processing: Background sweep compares last activity to an idle threshold; removes container; updates session status.

Outputs: Freed host resources; `UserLabSession.status=expired`.

Priority: Should

Acceptance Criteria: A session with no WebSocket activity beyond the configured threshold has its container removed and status updated without manual intervention.

Requirement ID: FR-09 **Title:** Author and Publish Lab Content

Description: The system shall let a content author publish a new lab by adding files under `labs/<track>/` and running a sync script, without application code changes.

Inputs: `lab.yaml`, step Markdown files, `*.check.sh` scripts.

Processing: `scripts/sync-labs.ts` parses lab content and upserts `Track/Lab/LabStep` rows via Prisma.

Outputs: New or updated catalog entries visible to students.

Priority: Must

Acceptance Criteria: A new, correctly formatted lab directory appears in the catalog after running the sync script, with no code deployment required.

Requirement ID: FR-10 **Title:** Privileged (DinD) Sandbox Support

Description: The system shall support a distinct, narrower-trust "privileged" container profile for labs that require a nested Docker daemon.

Inputs: `Lab.privileged` flag.

Processing: `terminal-server` branches `container-spawn` options to grant the minimum additional privilege needed for nested Docker, without mounting the host socket.

Outputs: A working nested Docker daemon inside an otherwise ephemeral, resource-capped container.

Priority: Should

Acceptance Criteria: A DinD lab's steps (e.g., `docker run` inside the sandbox) complete successfully; the host Docker socket is never reachable from inside the sandbox.

Requirement ID: FR-11 **Title:** Student Dashboard

Description: The system shall display a dashboard summarizing a student's total points and completed labs.

Inputs: User ID (from session).

Processing: Aggregate `UserStepProgress` and `User.points` for the current user.

Outputs: Rendered dashboard view.

Priority: Could

Acceptance Criteria: Dashboard totals match the sum of completed, points-bearing steps in the database for that user.

Requirement ID: FR-12 **Title:** Session Result Reporting (Internal)

Description: The system shall accept step-result reports only from `terminal-server`, authenticated by a shared secret, on a dedicated internal endpoint.

Inputs: Shared secret header, session ID, step ID, pass/fail.

Processing: Verify shared secret; write `UserStepProgress`; award points.

Outputs: 200 OK with updated progress, or 401 if the secret is missing/wrong.

Priority: Must

Acceptance Criteria: Calls without the correct shared secret are rejected; calls with it persist progress exactly once per (session, step) pair.

4 Non-Functional Requirements

4.1 Performance

Terminal keystroke round-trip latency shall remain low enough for interactive use (target: under 150ms end-to-end on a local/LAN deployment). Step validation shall complete and report a result within a few seconds for validation scripts of the size used in this project's labs. The catalog page shall load in under 1 second under nominal load.

4.2 Security

All Docker socket access shall be confined to terminal-server. Full requirements are detailed in Section 9.

4.3 Reliability

Loss of a single sandbox container shall not corrupt persisted progress, since progress is written only after an explicit, successful validation report. A terminal-server restart shall not corrupt app-backend's stored state.

4.4 Availability

Within the single-VM class-project scope, the system targets best-effort availability during scheduled class/demo hours; no formal uptime SLA applies. Session state is recoverable from PostgreSQL after a service restart.

4.5 Usability

The lab page layout (terminal left, instructions and Check action right) shall require no explanation for a student already familiar with common hands-on-lab platforms such as KodeKloud.

4.6 Maintainability

Lab content shall be addable or editable purely as version-controlled files (`lab.yaml`, step Markdown, `*.check.sh`), without modifying application code, so content and platform code evolve independently.

4.7 Portability

The frontend shall run unmodified on any evergreen Chromium-, Gecko-, or WebKit-based desktop browser with WebSocket support. Backend services target any Linux host with Docker Engine and PostgreSQL available, without OS-specific code paths.

5 External Interface Requirements

5.1 User Interface

A responsive web UI providing: a catalog page (tracks and labs), a track detail page, a split-pane lab page (terminal left; Markdown step panel and Check button right), and a dashboard. All interaction is through a standard browser; no native client is required.

5.2 Hardware Interface

None directly; the system relies on the host VM's CPU, memory, and container runtime. No specialized hardware is required.

5.3 Software Interface

- **app-backend REST API:** /auth (signup, login), /tracks, /tracks/:slug, /labs/:slug, /labs/:slug/start, /internal/progress (shared-secret protected).
- **terminal-server WebSocket API:** /term upgrade endpoint carrying binary terminal frames plus JSON control frames (resize, check_step).
- **Docker Engine API:** consumed via dockerode for container spawn, exec, and teardown.
- **PostgreSQL:** accessed exclusively by app-backend through Prisma ORM.

5.4 Communication Interface

REST calls use HTTPS/JSON in a production deployment; the terminal uses one persistent WebSocket per active session, carrying both binary I/O and JSON control messages.

6 Data Requirements

6.1 Entities

User, Track, Lab, LabStep, UserLabSession, UserStepProgress.

6.2 Attributes

- **User:** id, email (unique), passwordHash, displayName, points, createdAt.
- **Track:** id, slug (unique), name, description, icon.
- **Lab:** id, trackId (FK), slug (unique), title, description, difficulty, durationMinutes, tags, orderInTrack, imageRef, privileged.
- **LabStep:** id, labId (FK), stepNumber, title, contentMarkdown, validationScript.
- **UserLabSession:** id, userId (FK), labId (FK), containerId, status, startedAt, endedAt.
- **UserStepProgress:** id, sessionId (FK), stepId (FK), status, completedAt.

6.3 Relationships

- Track (1) – (*) Lab
- Lab (1) – (*) LabStep
- User (1) – (*) UserLabSession
- Lab (1) – (*) UserLabSession
- UserLabSession (1) – (*) UserStepProgress
- LabStep (1) – (*) UserStepProgress

6.4 Validation Rules

- User.email must be unique and a syntactically valid email address.
- Track.slug and Lab.slug must be unique, URL-safe strings.
- (Lab.trackId, Lab.orderInTrack) should not collide within a track (enforced at content-authoring time via the sync script).

- (`LabStep.labId`, `LabStep.stepNumber`) must be unique – enforced by a database composite unique constraint.
- (`UserStepProgress.sessionId`, ... `stepId`) must be unique – enforced by a database composite unique constraint, preventing duplicate progress rows for the same step.
- `Lab.difficulty` must be one of {beginner, intermediate, advanced}; `UserLabSession.status` must be one of {running, completed, expired, terminated}; `UserStepProgress.status` must be one of {pending, completed} – enforced via database enum types.

6.5 Database Requirements

PostgreSQL is the system of record, accessed exclusively by app-backend through Prisma ORM, which also manages schema migrations. Foreign keys enforce referential integrity across all relationships above; composite unique indexes back the validation rules listed. An index on (`trackId`, `orderInTrack`) supports ordered lab listing per track, and an index on (`userId`, `labId`) supports fast session lookups.

7 Use Cases

UC-01: User Registration

Actor(s): Student.

Preconditions: Actor has a valid, unregistered email address.

Main Flow:

1. Actor submits email, password, and display name.
2. System validates the email format and checks uniqueness.
3. System hashes the password and creates a `User` record.
4. System returns an authenticated session and redirects to the catalog.

Alternate Flow: If the email is already registered, the system rejects the request with a clear, non-account-enumerating error and the actor may retry with different credentials.

Postconditions: A new `User` row exists; the actor is authenticated.

UC-02: User Login

Actor(s): Student.

Preconditions: Actor has a registered account.

Main Flow:

1. Actor submits email and password.
2. System looks up the user and verifies the password hash.
3. System issues an authenticated session.

Alternate Flow: If credentials are invalid, the system returns a generic authentication error without revealing whether the email exists.

Postconditions: Actor holds a valid authenticated session.

UC-03: Browse Catalog and Start a Lab

Actor(s): Student.

Preconditions: Actor is authenticated.

Main Flow:

1. Actor views the track catalog and selects a track.

2. System lists that track's labs in authored order.
3. Actor selects a lab and clicks Start.
4. System provisions an ephemeral sandbox container from the lab's image and issues a scoped JWT.
5. Frontend opens a WebSocket terminal session using the JWT.

Alternate Flow: If container provisioning fails (e.g., host resource exhaustion), the system surfaces an error and does not create a `UserLabSession` in `running` state.

Postconditions: A running sandbox and an active `UserLabSession` exist for the actor.

UC-04: Interactive Terminal Session

Actor(s): Student; terminal-server (system actor).

Preconditions: A sandbox JWT has been issued (UC-03).

Main Flow:

1. Frontend opens a WebSocket to `/term` with the JWT.
2. terminal-server verifies the JWT and request Origin.
3. terminal-server execs an interactive shell in the session's container and bridges binary frames both ways.
4. Actor types commands and sees live output.

Alternate Flow: If the JWT is invalid, expired, or the Origin does not match, the upgrade is rejected before any container access.

Postconditions: Actor has a live shell inside their own sandbox container.

UC-05: Check Step (Auto-Validation)

Actor(s): Student; terminal-server (system actor).

Preconditions: Actor has an active terminal session on an unlocked step.

Main Flow:

1. Actor performs the step's intended actions in the terminal.
2. Actor clicks Check, sending a `check_step` frame.
3. terminal-server execs the step's validation script inside the container.
4. On success, terminal-server reports the result to app-backend via `/internal/progress`.
5. app-backend persists `UserStepProgress`, awards points, and the next step unlocks in the UI.

Alternate Flow: If the script exits non-zero, the system returns `passed:false`; the current step remains locked and the actor may retry after correcting their work.

Postconditions: On success, progress and points are durably persisted and the next step is unlocked; on failure, no state changes.

UC-06: Idle Session Reaping

Actor(s): terminal-server (system actor).

Preconditions: A `UserLabSession` is in `running` status with no recent activity.

Main Flow:

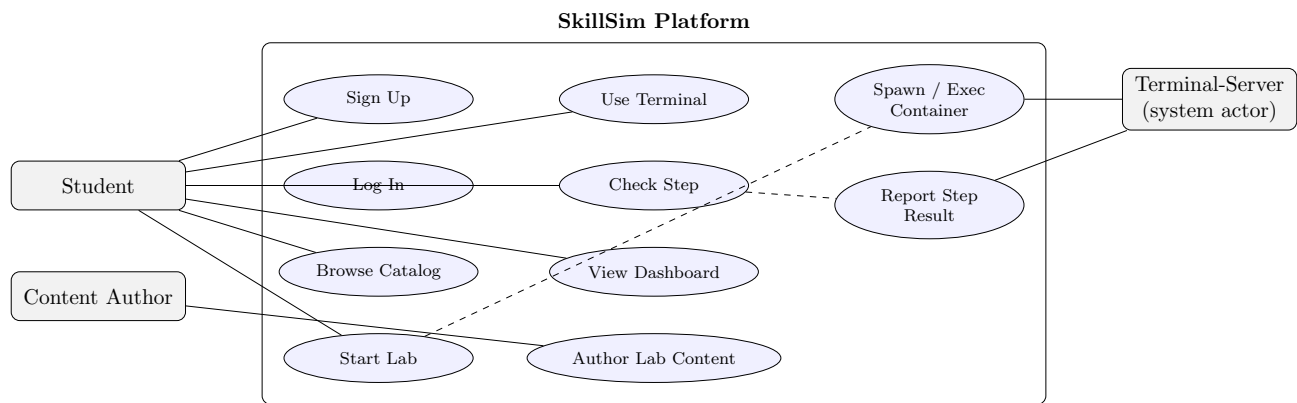
1. A background sweep identifies sessions idle past the configured threshold.
2. terminal-server removes the associated sandbox container.
3. terminal-server updates the session's status to `expired`.

Alternate Flow: If container removal fails, the sweep logs the failure and retries on the next cycle rather than marking the session expired prematurely.

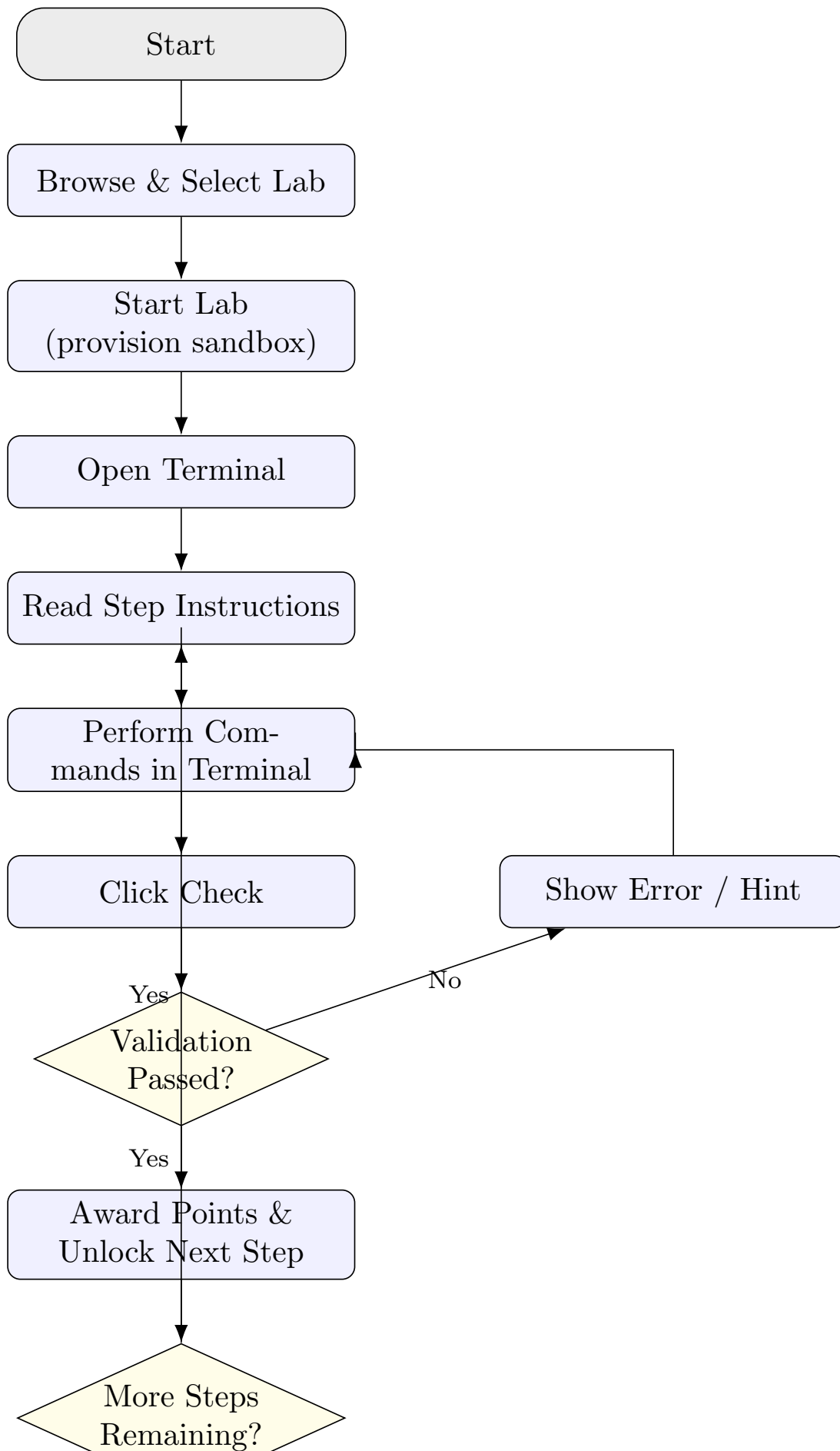
Postconditions: Host resources are freed; the session no longer accepts terminal input.

8 Analysis Models

8.1 Use Case Diagram

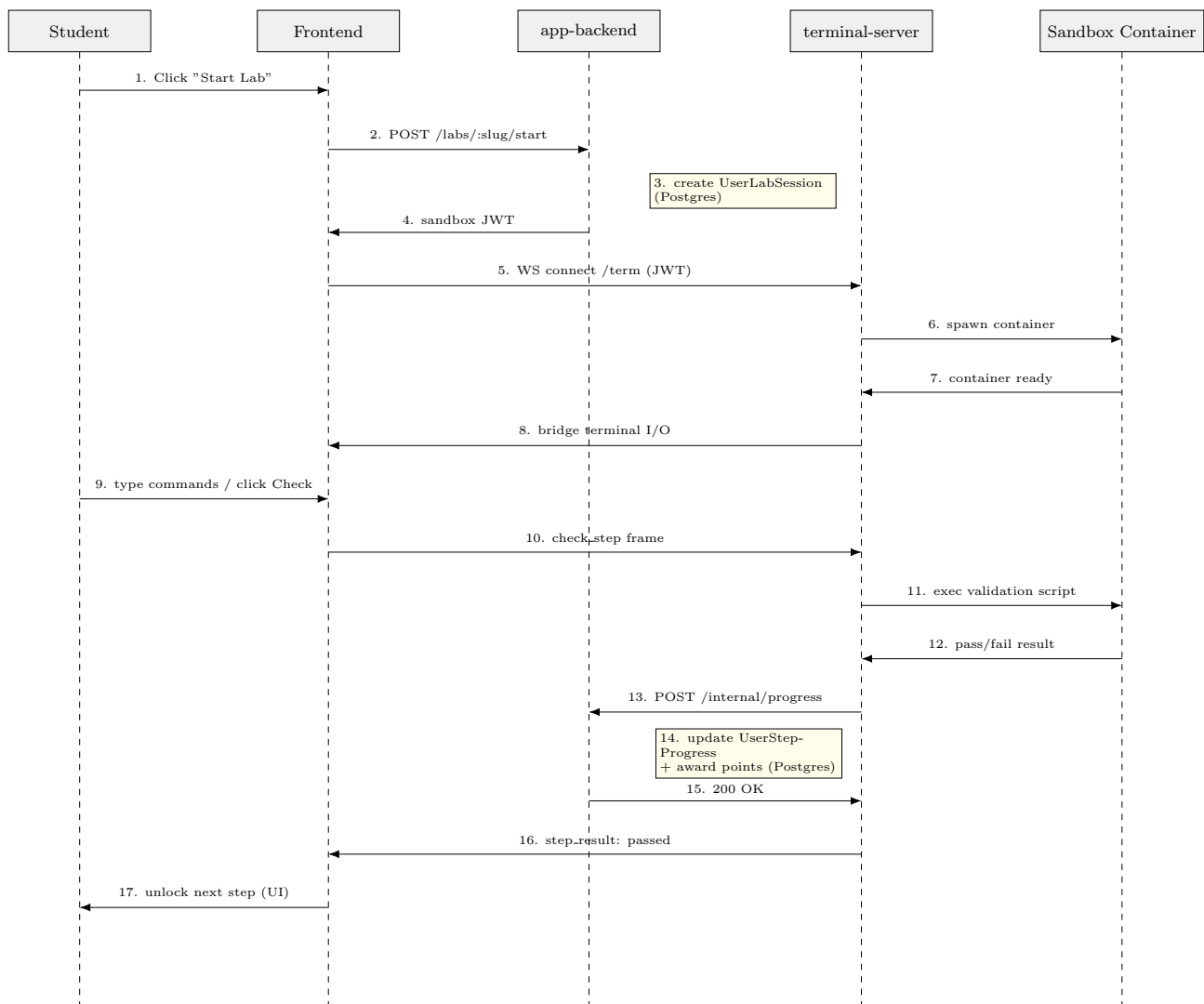


8.2 Activity Diagram

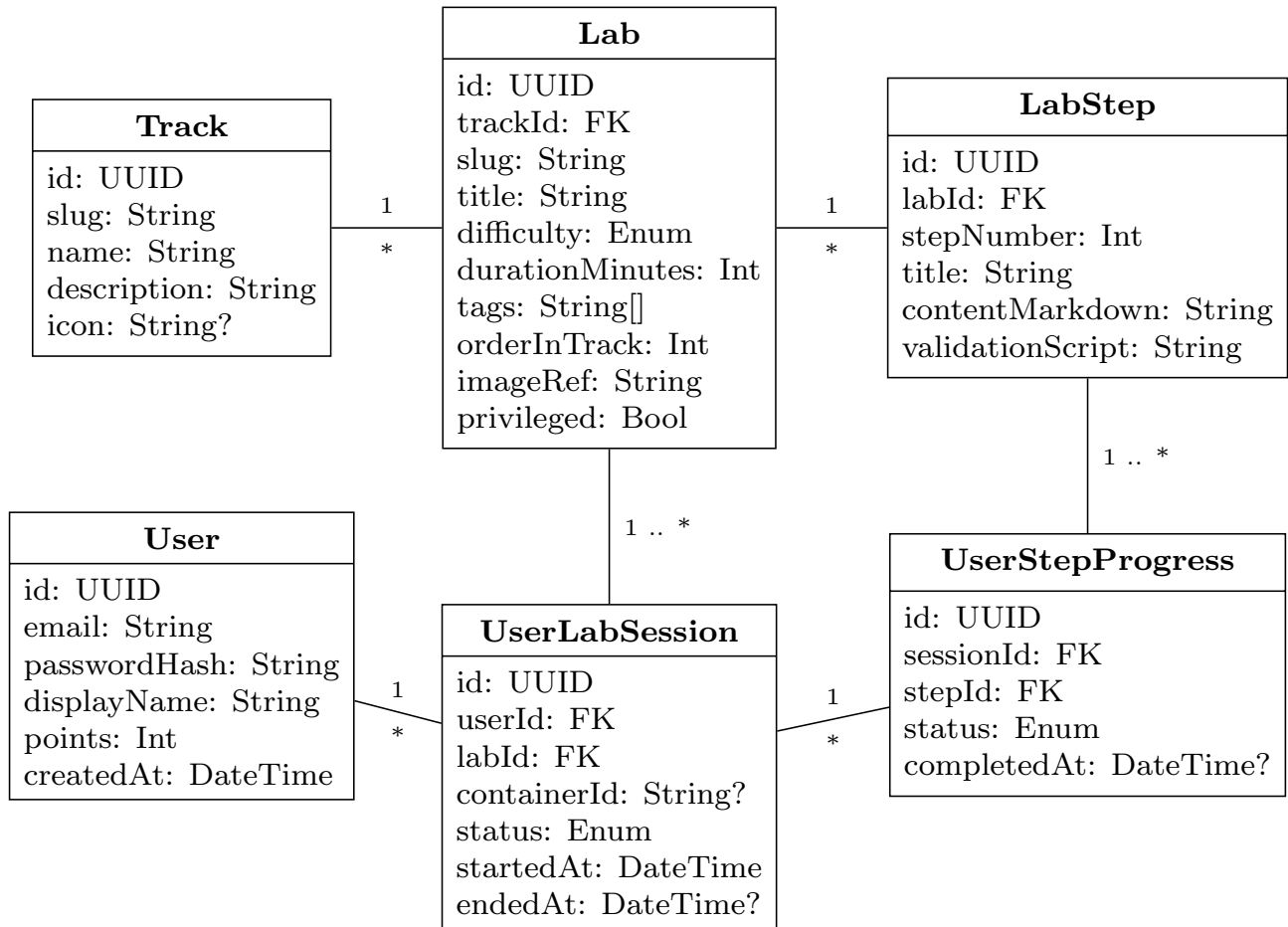


8.3 Sequence Diagram

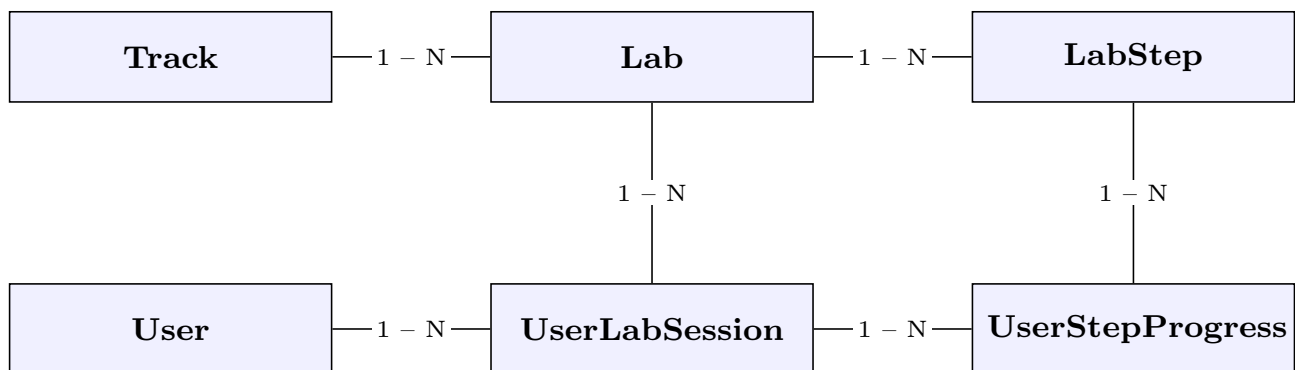
Flow: student starts a lab, opens a terminal, and successfully checks a step.



8.4 Class Diagram



8.5 ER Diagram / Data Model



9 Security Requirements

9.1 Authentication

The system shall authenticate students via email and a bcrypt- or Argon2-hashed password before issuing any session credential. Sandbox/terminal access shall additionally require a short-lived JWT

scoped to `sandbox:attach` for one specific session, separate from the student’s login session.

9.2 Authorization

A student’s JWT shall authorize access only to the sandbox container of their own active session; terminal-server shall verify the session ID embedded in the JWT against the requested container before attaching. The `/internal/progress` endpoint shall accept requests only from terminal-server, authenticated by a shared secret never exposed to the browser.

9.3 Input Validation

All REST and WebSocket control-frame inputs shall be validated server-side (type, length, and allow-listed values) before use, preventing SQL injection (mitigated structurally by Prisma’s parameterized queries) and cross-site scripting in any rendered Markdown step content.

9.4 Encryption

All communication shall use HTTPS/TLS in a production deployment, including the WebSocket terminal connection (WSS). Password hashes shall use bcrypt or Argon2 with a per-user salt; JWTs shall be signed with a server-held secret not exposed to clients.

9.5 Logging and Auditing

The system shall log lab session starts/ends, container spawn/teardown events, and step check results (pass/fail, timestamp, user, step) to support debugging and post-hoc auditing of student progress.

9.6 Session Management

Sandbox JWTs shall be short-lived and single-session-scoped, rejected immediately on session expiry, completion, or termination. Idle sessions shall be reaped per FR-08, releasing both the container and the session’s implicit terminal-access authorization.

9.7 Backup and Recovery

PostgreSQL, as the system of record, shall be backed up on a regular schedule (recommended: nightly logical dump) sufficient to recover user accounts and progress after host failure. Sandbox containers are intentionally ephemeral and are not backed up – only their pass/fail results, persisted to PostgreSQL, need survive a container’s loss.

10 Risk Analysis

ID	Risk	Likelihood	Impact	Mitigation
R-01	Host VM resource exhaustion from too many concurrent sandbox containers.	Medium	High	Idle-reap (FR-08) plus a per-host concurrency cap on active sessions.

ID	Risk	Likelihood	Impact	Mitigation
R-02	Docker socket compromise via a terminal-server bug, allowing host-level container escape.	Low	High	Strict input validation on control frames; terminal-server is the only process with socket access; default profile drops all capabilities and disables networking.
R-03	Privileged DinD container escape to the host.	Low	High	Privileged sandboxes remain ephemeral, resource-capped, and never mount the host's Docker socket; used only when <code>Lab.privileged</code> is explicitly true.
R-04	Five-week schedule slip against the fixed class-project timeline.	Medium	Medium	Locked scope (Section 2.5); MoSCoW-prioritized requirements so Should/Could items can slip first.
R-05	Validation script false pass or false fail, undermining trust in auto-validation.	Low	Medium	Peer review of each <code>.check.sh</code> script during authoring; manual smoke test per lab before publishing.
R-06	Single VM is a single point of failure for the whole platform.	Medium	Medium	Accepted risk for class-project scope; regular PostgreSQL backups (Section 9.7) limit data loss if the host fails.
R-07	Content author error (e.g., malformed <code>lab.yaml</code>) breaks the sync script or publishes a broken lab.	Medium	Low	Sync script validates required fields before writing to the database; broken labs fail sync rather than publishing partially.

11 Requirement Traceability Matrix (RTM)

Business Requirement	Functional Test Case Req.	
Students can create and access their own account.	FR-01, FR-02	TC-01, TC-02
Students can discover available learning content.	FR-03	TC-03
Students can begin a hands-on exercise in an isolated environment.	FR-04	TC-04
Students can interact with the exercise environment in real time.	FR-05	TC-05

Business Requirement	Functional Test Case Req.	
Student progress is graded automatically and objectively.	FR-06, FR-07	TC-06, TC-07
Idle resources are reclaimed without operator intervention.	FR-08	TC-08
Instructional team can add content without engineering involvement.	FR-09	TC-09
Advanced labs (e.g., Docker-in-Docker) are supported safely.	FR-10	TC-10
Students can see their overall standing.	FR-11	TC-11
Progress-reporting channel is trusted and tamper-resistant.	FR-12	TC-12

12 Acceptance Criteria

The system shall be accepted as meeting these requirements when the following measurable conditions are demonstrated end-to-end, not merely unit-tested in isolation:

1. A new user can sign up, log in, browse a track, and open a lab, with registration and login each completing in under 2 seconds (FR-01, FR-02).
2. Starting a lab provisions a real, isolated container and opens a working interactive terminal within a few seconds (FR-04, FR-05).
3. Running a step's intended commands, then clicking Check, executes that step's real validation script inside that same container and returns a result over the WebSocket (FR-06).
4. On a passing check, the next step unlocks in the UI and the corresponding `UserStepProgress` row and awarded points are durably visible in PostgreSQL afterward (FR-07).
5. A privileged (DinD) lab completes the same flow using a nested Docker daemon inside its own sandbox, with the host Docker socket never exposed to it (FR-10).
6. Idle sessions are reaped and their containers removed without requiring student action, within one sweep cycle of exceeding the idle threshold (FR-08).
7. A new lab can be added by a content author purely by adding files under `labs/` and running the sync script, with no application code change (FR-09).
8. Calls to `/internal/progress` without the correct shared secret are rejected 100% of the time in testing (FR-12, Section 9.2).

13 Appendices

13.1 Glossary

See Section 1.3 (Definitions, Acronyms and Abbreviations).

13.2 References

See Section 1.4.

13.3 Revision History

Version	Date	Author	Description
0.1	2026-07-28	Team SkillSim	Initial architecture and scaffold locked in; informal design notes only.
1.0	2026-08-13	Team SkillSim	First formal SRS, IEEE 29148 structure: functional/non-functional requirements, interfaces, data model, use cases, analysis models, security requirements, risk analysis, RTM, and acceptance criteria.